



## Assignment: Huffman Coding in C++

Tutorial on Huffman Encoding (Basics and theory only)

**DO NOT PLAGIARIZE. DO NOT USE AI TO GENERATE FULL CODE.**

### Objective

Implement a complete Huffman coding compressor/decompressor for text strings. The program must read a string, compute character frequencies, build a Huffman tree, generate prefix codes, encode the string into a binary string (characters '0' and '1'), decode it back, and display the compression ratio.

### Provided Skeleton

You are given an incomplete C++ program below. Your task is to fill in the missing functions step by step. All modifications should be confined to the sections marked with `/* TODO ... */`. The `main()` function is complete and must not be altered.

```
1 #include <iostream>
2 #include <string>
3 #include <map>
4 #include <queue>
5 #include <vector>
6 using namespace std;
7
8 // Node of the Huffman tree
9 struct Node {
10     char ch;
11     int freq;
12     Node *left, *right;
13     Node(char c, int f) : ch(c), freq(f), left(nullptr), right(nullptr) {}
14 };
15
16 // Comparator for the min-heap (priority queue)
17 struct Compare {
18     bool operator()(Node* a, Node* b) {
19         return a->freq > b->freq;    // smaller frequency = higher priority
20     }
21 };
22
23 // Step 1 - Build frequency table
24 map<char, int> buildFrequency(const string& text) {
25     /* TODO: count each character, return a map<char,int> */
26 }
27
28 // Step 2 - Build Huffman tree
29 Node* buildHuffmanTree(const map<char, int>& freqMap) {
30     /* TODO:
31         - Create a leaf Node for every character.
```

```

32     - Push all leaf nodes into a priority_queue<Node*, vector<Node*>,
        Compare>.
33     - While there is more than one node in the queue:
34         * Pop the two nodes with smallest frequencies.
35         * Create a new internal node with freq = sum of the two,
36           left = first popped, right = second popped.
37         * Push the new node back.
38     - Return the last remaining node (the root).
39     */
40 }
41
42 // Step 3 - Generate Huffman codes
43 void generateCodes(Node* root, const string& code, map<char, string>&
    huffmanCodes) {
44     /* TODO:
45         - If root is nullptr, return.
46         - If root is a leaf (left == nullptr && right == nullptr):
47             assign code to huffmanCodes[root->ch] (use "0" when tree has only
              one node).
48         - Otherwise:
49             traverse left with code + "0"
50             traverse right with code + "1"
51     */
52 }
53
54 // Step 4 - Encode the input string
55 string encode(const string& text, const map<char, string>& huffmanCodes) {
56     /* TODO: for each character c in text, append huffmanCodes.at(c) */
57 }
58
59 // Step 5 - Decode the binary string
60 string decode(const string& encoded, Node* root) {
61     /* TODO:
62         - Start at root.
63         - For each bit '0' go left, '1' go right.
64         - When a leaf is reached, append its character to the result and
          reset to root.
65         - Special case: if the tree consists of a single leaf, simply output
          that
66           character for every bit in the encoded string.
67     */
68 }
69
70 int main() {
71     string input;
72     cout << "Enter a string to compress: ";
73     getline(cin, input);
74
75     if (input.empty()) {
76         cout << "Empty string!" << endl;
77         return 0;
78     }
79
80     // Step 1 - Frequency table
81     map<char, int> freq = buildFrequency(input);
82
83     // Step 2 - Huffman tree

```

```

84 Node* root = buildHuffmanTree(freq);
85
86 // Step 3 - Code table
87 map<char, string> huffmanCodes;
88 generateCodes(root, "", huffmanCodes);
89
90 // Step 4 - Encode
91 string encoded = encode(input, huffmanCodes);
92
93 // Step 5 - Decode
94 string decoded = decode(encoded, root);
95
96 // Output
97 cout << "\nOriginal text : " << input << endl;
98 cout << "Encoded binary : " << encoded << endl;
99 cout << "Decoded text : " << decoded << endl;
100
101 // Compression stats (optional but interesting)
102 int originalBits = input.size() * 8; // assuming 8 bits per char
103 int compressedBits = encoded.size(); // length of the 0/1 string
104 cout << "\nOriginal size : " << originalBits << " bits" << endl;
105 cout << "Compressed size : " << compressedBits << " bits" << endl;
106 cout << "Compression ratio: "
107      << (double)compressedBits / originalBits * 100 << "%" << endl;
108
109 return 0;
110 }

```

Listing 1: Incomplete skeleton code (save as `huffman.cpp`)

## Your Tasks (Step-by-step Instructions)

1. **Frequency table** — Implement `buildFrequency`. The function receives a string and returns a `map<char,int>` where each key is a character and the value is the number of times it appears. *Hint:* Use a simple loop and the `[]` operator.

2. **Huffman tree** — Implement `buildHuffmanTree`. Follow the classic Huffman algorithm:

- Create a leaf `Node` for every character-frequency pair.
- Push all leaves into a `priority_queue<Node*, vector<Node*>, Compare>`.
- Repeatedly remove the two nodes with smallest frequencies, combine them into a new internal node, and push the new node back. Stop when one node remains.

*Note:* You may ignore memory deallocation for this assignment.

3. **Code generation** — Implement `generateCodes`. Recursively traverse the tree. When you reach a leaf, store the accumulated path in the `huffmanCodes` map. *Special case:* If the input contains only one distinct character, the tree will be a single leaf. In that case assign the code "0" to that character.

4. **Encoding** — Implement `encode`. For every character of the original text, look up its Huffman code and append it to the result string. *Hint:* Use `huffmanCodes.at(c)` to guarantee a code exists for every character.

5. **Decoding** — Implement `decode`. Start at the root. For each bit (character '0' or '1') move left or right. Whenever a leaf is reached, output the leaf's character and reset the current node to the

root. *Edge case:* If the root itself is a leaf (single unique character), simply repeat that character for every bit in the encoded string.

6. **Testing** — Once you complete the functions, compile and test your program. Example inputs and expected behaviours:

- Input: "hello" → Encoded binary string of varying length codes, decoded matches original.
- Input: "aaaa" → Encoded string "0000", decoded "aaaa".
- Input: "a" → Encoded "0", decoded "a".

7. **(Optional)** Compute the compression ratio in `main()` and verify it.

## Hints and Tips

- Keep everything simple: codes are stored as strings of '0' and '1'; you do **not** need to pack bits into actual bytes.
- The `priority_queue` requires a custom comparator. The provided `Compare` struct already implements a min-heap based on frequency.
- When combining two nodes, the order of left/right child does not affect correctness, but it will influence the code assignment. Any valid prefix-free code is acceptable.
- Recursion is the easiest way to generate codes: start with an empty string, append '0' for left child, '1' for right child.
- To handle the single-character case cleanly, you may assign "0" directly inside `generateCodes` when the root has no children.
- Memory management: you may leak nodes; it is acceptable for this exercise. If you want to be tidy, delete the tree at the end of `main()` (not required).
- The provided `main()` outputs the encoded string, the decoded string, and a rough compression ratio. Do not alter the `main()` function.

## Submission

Submit a single `.cpp` file containing your complete implementation. Make sure it compiles with a standard C++17 (or later) compiler and runs the `main()` exactly as given.